

Quantium Virtual Internship — Retail Strategy and Analytics — Task 1

Chip Category Analysis (Python adaptation of the R Markdown solution template)

Retail Analytics Team

31 July 2026

Contents

Introduction	2
Load required libraries and datasets	2
Exploratory data analysis	3
Examining transaction data	3
Examine PROD_NAME	3
Further examine PROD_NAME: word frequency check	3
Remove non-chip (salsa) products	4
Initial summary: check for nulls and outliers	4
Investigate the PROD_QTY outlier	4
Check other transactions from that customer	4
Transactions over time: checking for missing dates	6
Create pack size	7
Create brand name	7
Clean brand names	8
Examining customer data	9
Merge transaction data to customer data	9
Check for missing customer details	9
Save the merged dataset	9
Data analysis on customer segments	10
Total sales by LIFESTAGE and PREMIUM_CUSTOMER	10
Number of customers by LIFESTAGE and PREMIUM_CUSTOMER	10
Average packets bought per customer, by segment	10
Average price per unit, by segment	12
Statistical test: is the price difference significant?	12
Deep dive into specific customer segments	14
Brand preference: Mainstream - Young Singles/Couples	14
Preferred pack size: Mainstream - Young Singles/Couples	14
Conclusion and recommendations	17
Summary of findings	17
Recommendation to Julia	17

Introduction

This document works through Task 1 of the Quantum Virtual Internship: analysing one year of chip-category transaction data (QVI_transaction_data.xlsx) matched to customer loyalty profiles (QVI_purchase_behaviour.csv) to understand who buys chips and what drives sales, for the Category Manager's strategic review.

The official task template is written in R. This document follows the **same analysis structure and order** as that template, but every step is implemented in **Python** (pandas, numpy, scipy, matplotlib), since that is the tool used here. Each section shows the working code, the real output it produced, and the interpretation drawn from it.

Load required libraries and datasets

```
import pandas as pd
import numpy as np
import re
from scipy import stats
import matplotlib.pyplot as plt

transactionData = pd.read_excel("QVI_transaction_data.xlsx")
customerData = pd.read_csv("QVI_purchase_behaviour.csv")
```

Exploratory data analysis

Examining transaction data

```
print(transactionData.shape)
print(transactionData.dtypes)
print(transactionData[["PROD_NAME", "PROD_QTY", "TOT_SALES"]].head())
```

Output:

(264836, 8)

```
DATE           int64
STORE_NBR      int64
LYLTY_CARD_NBR int64
TXN_ID         int64
PROD_NBR       int64
PROD_NAME      object
PROD_QTY       int64
TOT_SALES      float64
```

		PROD_NAME	PROD_QTY	TOT_SALES
0	Natural Chip Compny	SeaSalt175g	2	6.0
1	CCs Nacho Cheese	175g	3	6.3
2	Smiths Crinkle Cut	Chips Chicken 170g	5	15.0
3	Smiths Chip Thinly	S/Cream&Onion 175g	3	8.4

DATE is stored as an integer (an Excel/CSV serial date, days since 30 Dec 1899), not a real date — this needs converting.

```
transactionData["DATE"] = pd.to_datetime(transactionData["DATE"], unit="D",
                                          origin="1899-12-30")
```

Examine PROD_NAME

```
print(transactionData["PROD_NAME"].nunique(), "unique product names")
print(transactionData["PROD_NAME"].value_counts().head(10))
```

Output: 114 unique product names. All look like packaged chip/snack products (e.g. Kettle Sensations Siracha Lime 150g, Smiths Crinkle Cut Original 170g), which is what we'd expect — but let's check systematically rather than by eye.

Further examine PROD_NAME: word frequency check

The template's suggested check: split every product name into words, strip out words containing digits or special characters (these are pack sizes, not useful for spotting non-chip products), and count how often each remaining word appears.

```
words = pd.Series(" ".join(transactionData["PROD_NAME"].unique()).split())
clean_words = words[~words.str.contains(r"[\d&]", regex=True)]
freq = clean_words.value_counts()
print(freq.head(15))
```

Output:

Chips	21	Chip	9
Smiths	16	Salsa	9
Crinkle	14	Doritos	9

Cut	14	Corn	8
Kettle	13	Pringles	8
Cheese	12	RRD	8
Salt	12	Chicken	7
Original	10	WW	7

Everything here is a legitimate chip brand, flavour, or descriptor word, **except “Salsa”** (9 occurrences) and “Dip” further down the list — these flag products that may not actually be chips.

Remove non-chip (salsa) products

```
transactionData["PACK_SIZE_TMP"] = (transactionData["PROD_NAME"]
                                     .str.extract(r"(\d+)\s*[gG]\b").astype(int))
is_salsa = transactionData["PROD_NAME"].str.contains("salsa", case=False)
print(transactionData.loc[is_salsa, ["PROD_NAME",
                                     "PACK_SIZE_TMP"]].drop_duplicates())
```

Output: 9 products contain “salsa” in the name — but checking their pack sizes shows only **7 of them are genuine dip tubs**, all sold at a 300g size that no real chip product in this dataset uses. The other 2 (Smiths...Tomato Salsa 150g, Red Rock Deli...Salsa & Mzzrlla 150g) are chip products at the standard 150g chip size, where “salsa” is just a flavour name, not a dip. Removing only the 7 true dips (rather than blanket-removing every row containing “salsa”) avoids throwing away 2 legitimate chip products:

```
is_dip = transactionData["PROD_NAME"].str.contains("salsa", case=False) & \
        (transactionData["PACK_SIZE_TMP"] == 300)
transactionData = transactionData.loc[~is_dip].drop(columns="PACK_SIZE_TMP").copy()
print(f"Removed {is_dip.sum()} rows across 7 dip products")
```

Output: Removed 15166 rows across 7 dip products

Initial summary: check for nulls and outliers

```
print(transactionData.isnull().sum().sum(), "total nulls")
print(transactionData[["PROD_QTY", "TOT_SALES"]].describe())
```

Output: 0 total nulls. No missing data anywhere. But PROD_QTY has a **max of 200** against a 75th percentile of 3 — a clear outlier worth investigating, since a normal shopping basket is 1-5 packets.

Investigate the PROD_QTY outlier

```
print(transactionData.loc[transactionData["PROD_QTY"] == 200])
```

Output: Two transactions, both **200 packets, \$650 total sales each**, both made by the same loyalty card: **226000**.

Check other transactions from that customer

```
print(transactionData.loc[transactionData["LYLTY_CARD_NBR"] == 226000])
```

Output: These two 200-packet transactions are the **only** transactions this customer made all year. This isn’t a household shopper — most likely a commercial buyer stocking up for an event

or resale. We remove this loyalty card from further analysis so it doesn't distort per-customer averages.

```
transactionData = transactionData.loc[
    transactionData["LYLTY_CARD_NBR"] != 226000].copy()
print(transactionData["PROD_QTY"].describe())
```

Output: Max PROD_QTY is now 5 — sensible.

Transactions over time: checking for missing dates

```
daily_counts = transactionData.set_index("DATE").resample("D")["TXN_ID"].count()
print(f"{daily_counts.shape[0]} unique dates found in the data")
```

Output: 364 unique dates found in the data. A full year (1 Jul 2018 to 30 Jun 2019) should have 365 days — one day is missing. Building the full date sequence and re-plotting confirms it:

```
full_range = pd.date_range("2018-07-01", "2019-06-30")
daily_counts = daily_counts.reindex(full_range, fill_value=0)
```

```
fig, ax = plt.subplots(figsize=(11, 4))
ax.plot(daily_counts.index, daily_counts.values)
ax.set_title("Transactions Over Time")
ax.set_ylabel("Number of transactions")
plt.show()
```

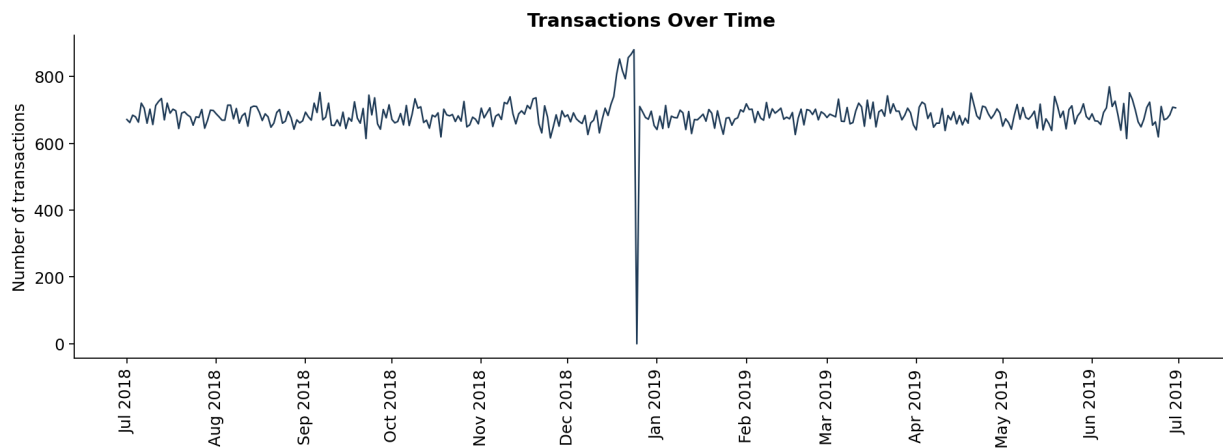


Figure 1: Transactions over the full year — a spike is visible in December, with a gap right at the end of the month.

There's a clear run-up in transactions through December, and a single-day gap right at the end of the month. Zooming into December confirms what it is:

```
dec = daily_counts["2018-12-01":"2018-12-31"]
fig, ax = plt.subplots(figsize=(9, 4))
ax.plot(dec.index, dec.values, marker="o")
ax.axvline(pd.Timestamp("2018-12-25"), linestyle="--")
plt.show()
```

Sales climb in the lead-up to Christmas and hit **zero on Christmas Day itself**, because stores are closed. This is an expected retail pattern, not a data error, so no correction is needed — the “missing” day is legitimate.

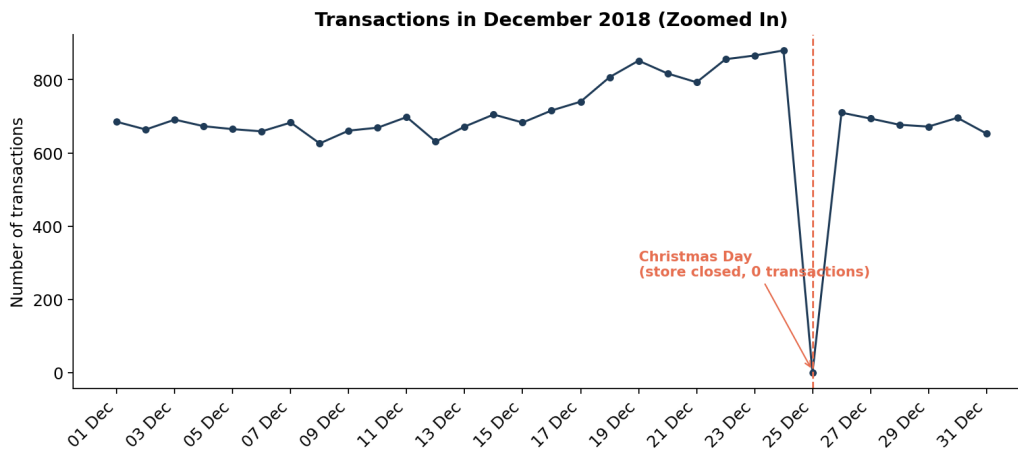


Figure 2: December 2018 zoomed in: transactions climb steadily toward Christmas, then drop to exactly zero on 25 December.

Create pack size

The template extracts pack size using `parse_number()` on `PROD_NAME`, which pulls the first number out of the string. The Python equivalent is a regex that finds a number immediately followed by “g” (grams) anywhere in the name (this matters because one product, “Kettle 135g Swt Pot Sea Salt”, has its size in the middle of the name rather than at the end):

```
transactionData["PACK_SIZE"] = (transactionData["PROD_NAME"]
                                .str.extract(r"(\d+)\s*[gG]\b").astype(int))
print(transactionData.groupby("PACK_SIZE").size().sort_index())
```

Output: Pack sizes range from a smallest of **70g** to a largest of **380g**, across 20 distinct sizes — all sensible values for a chip packet.

```
fig, ax = plt.subplots(figsize=(10, 4.5))
counts = transactionData["PACK_SIZE"].value_counts().sort_index()
ax.bar(counts.index.astype(str), counts.values)
ax.set_xlabel("Pack size (g)"); ax.set_ylabel("Number of transactions")
plt.xticks(rotation=90); plt.show()
```

Create brand name

```
transactionData["BRAND"] = transactionData["PROD_NAME"].str.split().str[0]
print(transactionData["BRAND"].value_counts())
```

Output (abridged):

Kettle	41288	RRD	11894	Red	5885	Snbts	1576
Smiths	28860	Infuzions	11057	Cheezels	4603	Burger	1564
Pringles	25102	WW	10320	CCs	4551	GrnWves	1468
Doritos	24962	Cobs	9693	Woolworths	4437	Sunbites	1432
Thins	14075	Tostitos	9471	Dorito	3185	NCC	1419
		Twisties	9454	Infzns	3144	French	1418
				Smith	2963		
				Cheetos	2927		

Checking this list, several rows are clearly the **same brand written two ways**: RRD / Red (Red Rock Deli), WW / Woolworths, Dorito / Doritos, Smith / Smiths, Infzns / Infuzions, Snbts / Sunbites,

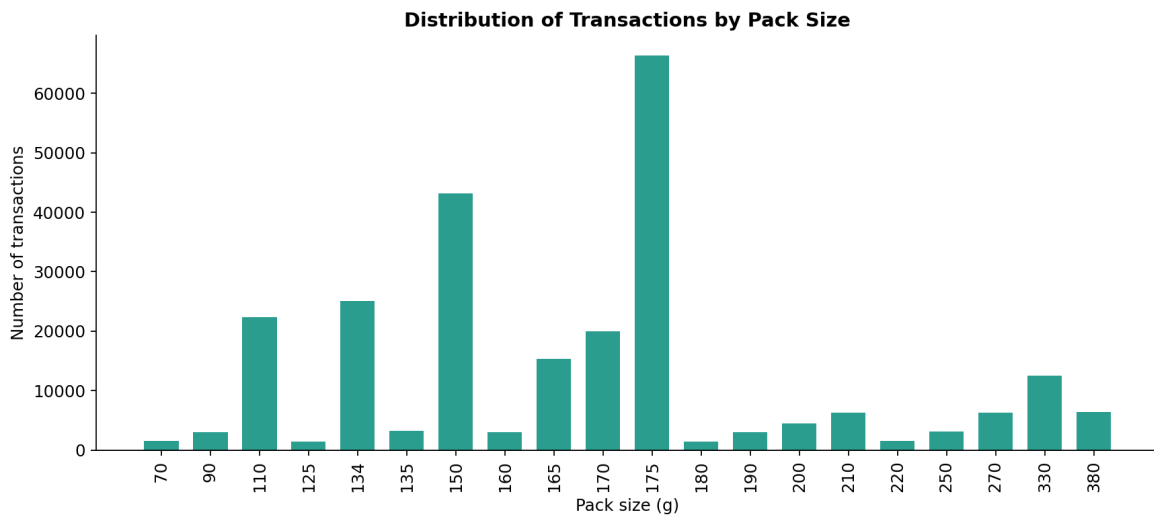


Figure 3: Transactions by pack size — 175g is the most common size by far, with a long tail of smaller specialty sizes.

NCC / Natural (Natural Chip Co), GrnWves / Grain (Grain Waves) — 8 pairs in total, all confirmed by reading the full product names for each variant.

Clean brand names

```
BRAND_MAP = {
    "RRD": "Red Rock Deli", "Red": "Red Rock Deli",
    "WW": "Woolworths",
    "Dorito": "Doritos",
    "Infzns": "Infuzions",
    "Smith": "Smiths",
    "Snbts": "Sunbites",
    "GrnWves": "Grain Waves", "Grain": "Grain Waves",
    "NCC": "Natural Chip Co", "Natural": "Natural Chip Co",
}
transactionData["BRAND"] = transactionData["BRAND"].replace(BRAND_MAP)
print(transactionData["BRAND"].nunique(), "distinct brands after cleaning")
```

Output: 20 distinct brands after cleaning (down from 28 raw first-words) — each one now maps to a single, real chip brand.

Examining customer data

```
print(customerData.shape)
print(customerData.isnull().sum().sum(), "nulls")
print(customerData.duplicated().sum(), "duplicate rows")
print(customerData["LIFESTAGE"].value_counts())
print(customerData["PREMIUM_CUSTOMER"].value_counts())
```

Output: (72637, 3), 0 nulls, 0 duplicate rows. Seven LIFESTAGE categories (Retirees 14,805 down to New Families 2,549) and three PREMIUM_CUSTOMER tiers (Mainstream 29,245, Budget 24,470, Premium 18,922) — all clean, consistent labels, no further cleaning required.

Merge transaction data to customer data

```
data = transactionData.merge(customerData, on="LYLTY_CARD_NBR", how="left")
print(len(transactionData), "->", len(data), "rows after merge")
```

Output: 249667 -> 249667 rows after merge — identical row count confirms the merge didn't create duplicates.

Check for missing customer details

```
print(data["LIFESTAGE"].isnull().sum(), "unmatched rows")
```

Output: 0 unmatched rows. Every transaction found a matching customer profile.

Save the merged dataset

```
data.to_csv("QVI_data.csv", index=False)
```

Data exploration is now complete. Final cleaned dataset: **249,667 transactions, 475,909 packets sold, \$1,819,778.40 total sales**, across **71,517 unique customers**.

Data analysis on customer segments

Four metrics describe chip-buying behaviour by segment: total sales, number of customers, average packets bought per customer, and average price paid per packet — each split by LIFESTAGE and PREMIUM_CUSTOMER.

Total sales by LIFESTAGE and PREMIUM_CUSTOMER

```
sales = data.pivot_table(index="LIFESTAGE", columns="PREMIUM_CUSTOMER",
                          values="TOT_SALES", aggfunc="sum")
# ... grouped bar chart of `sales` ...
```

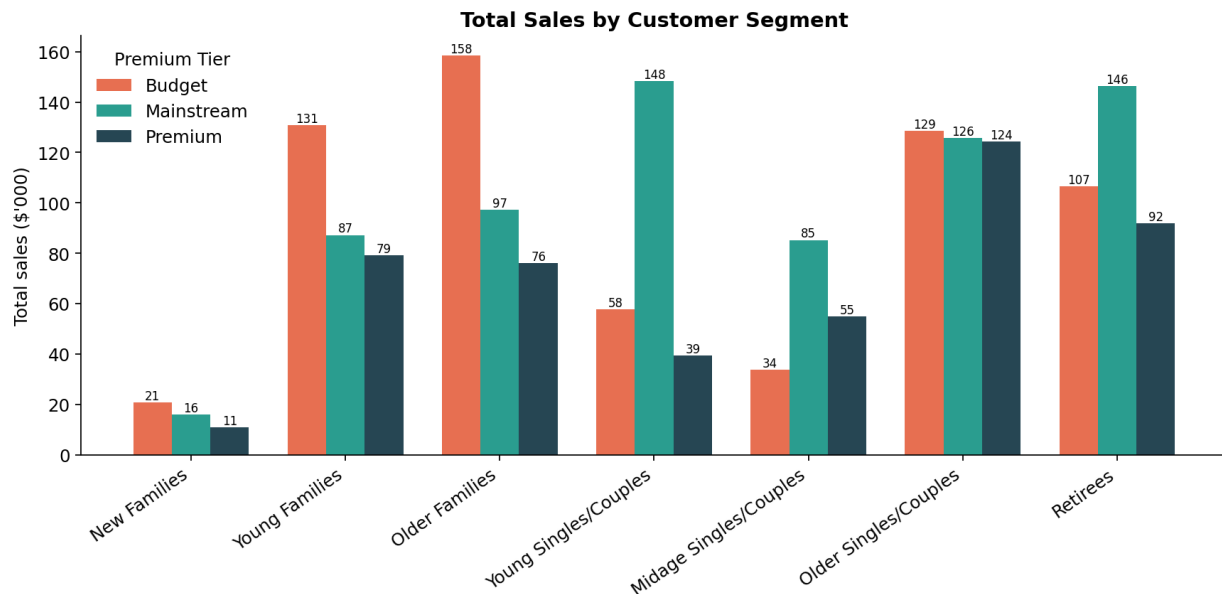


Figure 4: Total sales by customer segment.

Sales are coming mainly from **Budget - Older Families** (\$158,380), **Mainstream - Young Singles/Couples** (\$148,337), and **Mainstream - Retirees** (\$146,329).

Number of customers by LIFESTAGE and PREMIUM_CUSTOMER

```
custs = data.groupby(["LIFESTAGE", "PREMIUM_CUSTOMER"])["LYLTY_CARD_NBR"]
        .nunique().unstack()
```

There are far more **Mainstream - Young Singles/Couples** (7,930) and **Mainstream - Retirees** (6,382) customers than any other segment/tier combination. This is a major reason their total sales are high — but it does **not** explain Budget - Older Families' high sales, whose customer count (4,622) is unremarkable.

Average packets bought per customer, by segment

```
units_per_cust = data.groupby(["LIFESTAGE", "PREMIUM_CUSTOMER"]).apply(
    lambda g: g["PROD_QTY"].sum() / g["LYLTY_CARD_NBR"].nunique())
```

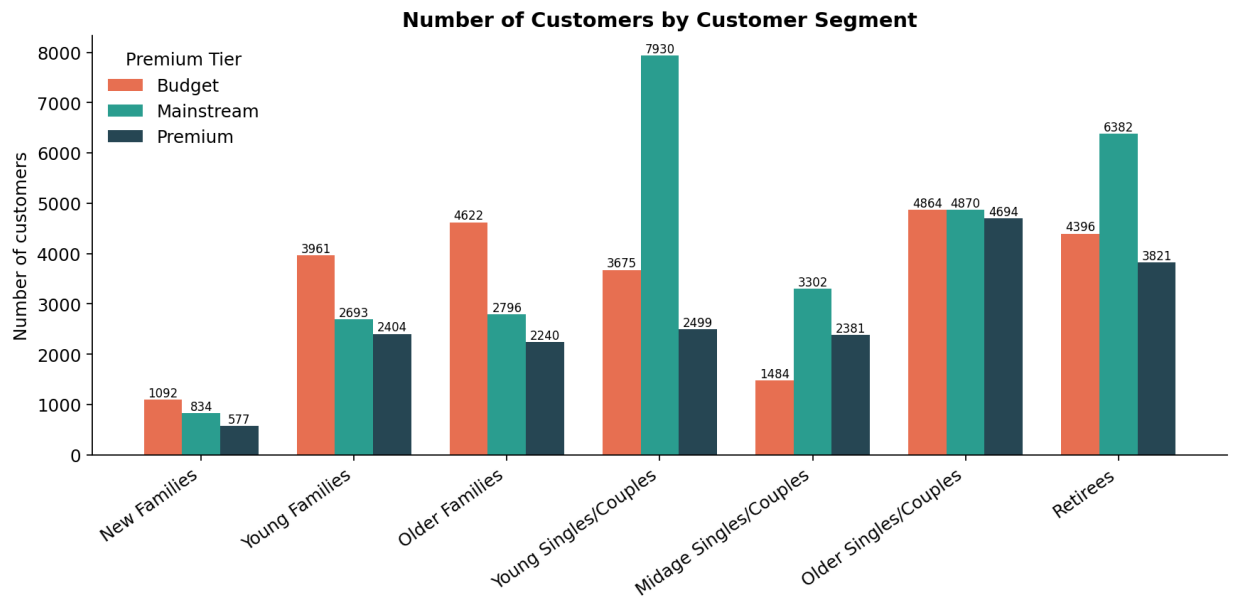


Figure 5: Number of customers by segment.

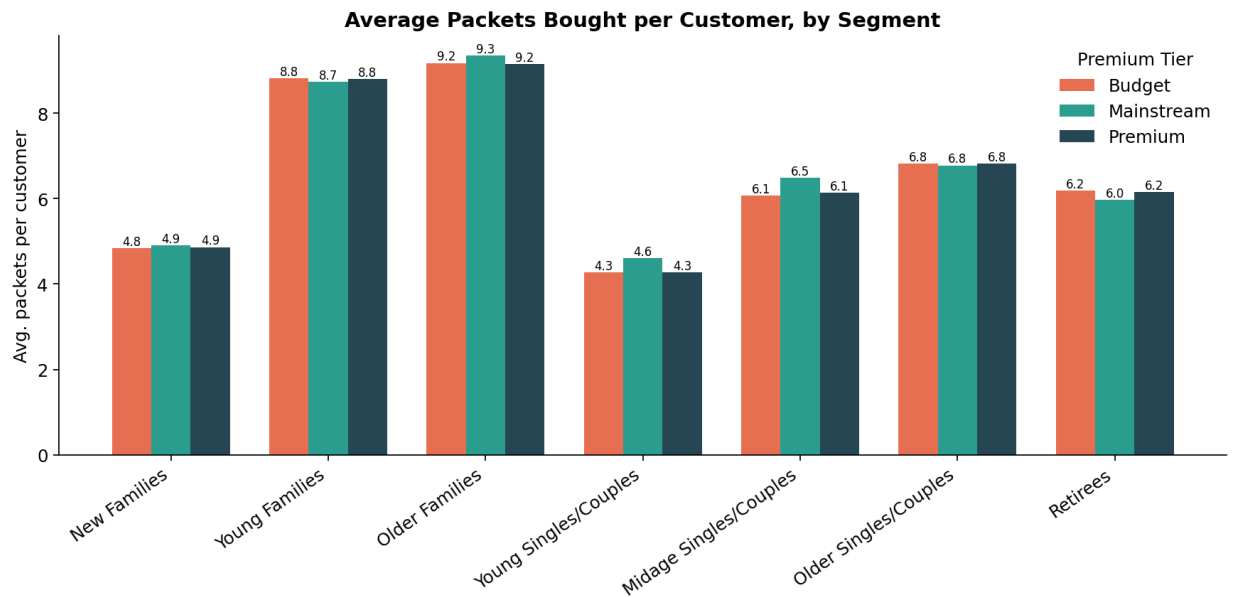


Figure 6: Average packets per customer, by segment.

Older and Young Families buy noticeably more packets per customer (8.7-9.3, regardless of Budget/Mainstream/Premium tier) than any other lifestage. This – not customer count – is what explains Budget - Older Families’ high total sales. Singles/Couples segments buy far fewer packets per customer (4.3-6.8).

Average price per unit, by segment

```
price_per_unit = data.groupby(["LIFESTAGE", "PREMIUM_CUSTOMER"]).apply(
    lambda g: g["TOT_SALES"].sum() / g["PROD_QTY"].sum())
```

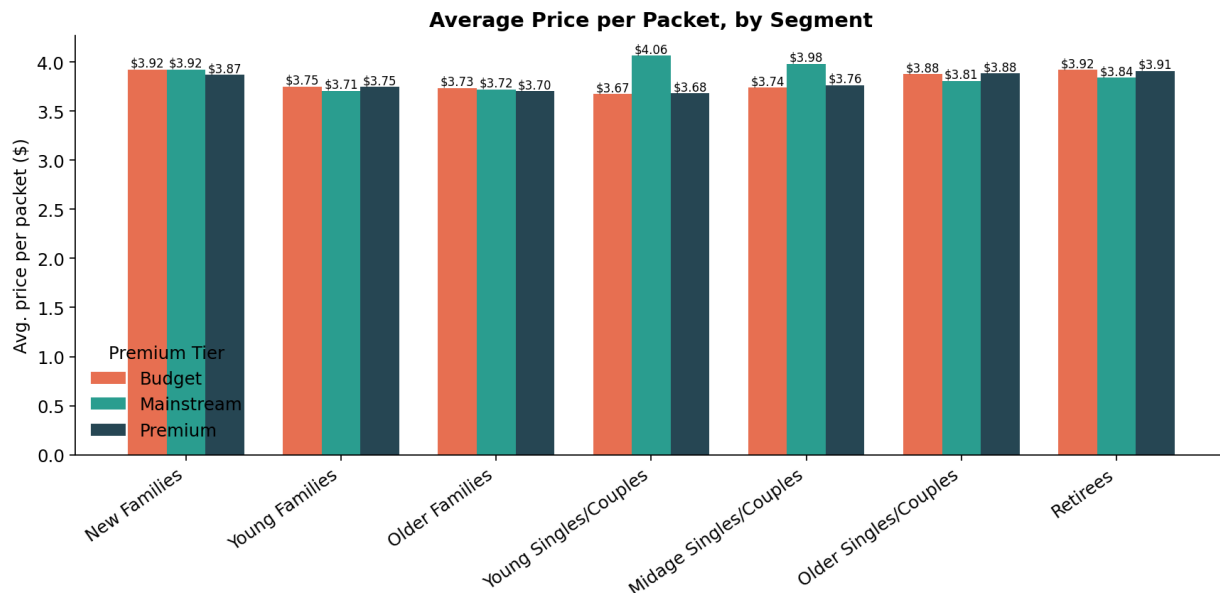


Figure 7: Average price per packet, by segment.

Mainstream young singles/couples and midage singles/couples are willing to pay more per packet of chips than their Budget or Premium counterparts in the *same* lifestage (\$4.06 and \$3.98 vs. \$3.67-\$3.76). This may be because premium shoppers skew toward healthier snacks and buy chips mainly for entertaining rather than personal consumption, while Mainstream singles/couples buy chips more habitually and are less price-sensitive doing so. Notably, this pattern does **not** hold for Retirees, where Mainstream pays about the same as Budget/Premium — so the “pays more” story is specific to younger/midage singles and couples, not a general Mainstream effect.

As the difference isn’t large in absolute terms, we check whether it’s statistically significant.

Statistical test: is the price difference significant?

```
sub = data[data["LIFESTAGE"].isin(["YOUNG SINGLES/COUPLES", "MIDAGE SINGLES/COUPLES"])]
sub["unit_price"] = sub["TOT_SALES"] / sub["PROD_QTY"]
mainstream = sub.loc[sub["PREMIUM_CUSTOMER"]=="Mainstream", "unit_price"]
other       = sub.loc[sub["PREMIUM_CUSTOMER"]!="Mainstream", "unit_price"]

t_stat, p_val = stats.ttest_ind(mainstream, other, equal_var=False)
print(f"t = {t_stat:.4f}, p = {p_val:.6e}")
print(f"Mainstream mean = ${mainstream.mean():.4f}, Other mean = ${other.mean():.4f}")
```

Output: $t = 38.1104$, $p = 0.000000e+00$ (mainstream mean = \$4.0278 vs. other mean = \$3.6918, $n = 30,905$ vs $27,104$).

The t-test results in a p-value of **effectively 0 ($p < 0.0001$)**, i.e. the unit price for Mainstream, young and midage singles and couples **IS** significantly higher than that of Budget or Premium, young and midage singles and couples — a 9.1% premium that is not due to chance.

Deep dive into specific customer segments

Mainstream - Young Singles/Couples stands out as a priority segment: the single largest customer base of any segment (7,930 customers, 11.1% of all customers), substantial total sales (\$148,337), and the statistically-confirmed price premium above. We dive into what this segment specifically buys.

Brand preference: Mainstream - Young Singles/Couples

```
data["is_target"] = ((data["PREMIUM_CUSTOMER"] == "Mainstream") &
                     (data["LIFESTAGE"] == "YOUNG SINGLES/COUPLES"))

target_pct = data[data["is_target"]].groupby("BRAND")["PROD_QTY"].sum()
target_pct = target_pct / target_pct.sum() * 100
rest_pct = data[~data["is_target"]].groupby("BRAND")["PROD_QTY"].sum()
rest_pct = rest_pct / rest_pct.sum() * 100

affinity = (target_pct / rest_pct).sort_values(ascending=False)
print(affinity)
```

Output (affinity index; 1.00 = same share of packets as everyone else):

Tyrrells	1.233	Infuzions	1.138	Cheetos	0.669
Twisties	1.224	Thins	1.064	Red Rock Deli	0.662
Doritos	1.220	Grain Waves	1.053	Natural Chip Co	0.638
Kettle	1.201	Cheezeels	0.968	CCs	0.594
Tostitos	1.201	Smiths	0.766	Sunbites	0.507
Pringles	1.192	French	0.688	Woolworths	0.490
Cobs	1.148			Burger	0.445

We can see that:

- **Mainstream young singles/couples over-index clearly on Tyrrells, Twisties, Doritos, Kettle, Tostitos, and Pringles** — all 19-23% more likely to be bought by this segment than by everyone else.
- They **under-index heavily on Woolworths (own-brand), Burger Rings, Sunbites, CCs, Natural Chip Co, Red Rock Deli, and Cheetos** — all bought roughly 33-55% less than their share among other customers.
- The pattern looks like a preference for **recognised, mainstream snack brands** over budget/home-brand and lower-profile options, consistent with this segment's demonstrated willingness to pay a premium price per packet (Section 4.5).

Preferred pack size: Mainstream - Young Singles/Couples

```
target_pct = data[data["is_target"]].groupby("PACK_SIZE")["PROD_QTY"].sum()
target_pct = target_pct / target_pct.sum() * 100
rest_pct = data[~data["is_target"]].groupby("PACK_SIZE")["PROD_QTY"].sum()
rest_pct = rest_pct / rest_pct.sum() * 100
size_affinity = (target_pct / rest_pct).sort_values(ascending=False)
print(size_affinity)
```

```
# check: is pack size independent of brand, or just a re-labelling of it?
print(data.groupby("BRAND")["PACK_SIZE"].nunique().sort_values())
```

Output: Individual sizes *look* like they vary a lot — 270g and 380g are both around 1.26-1.27x, while 220g and 70g sit around 0.44-0.48x. But checking PACK_SIZE against BRAND shows why:

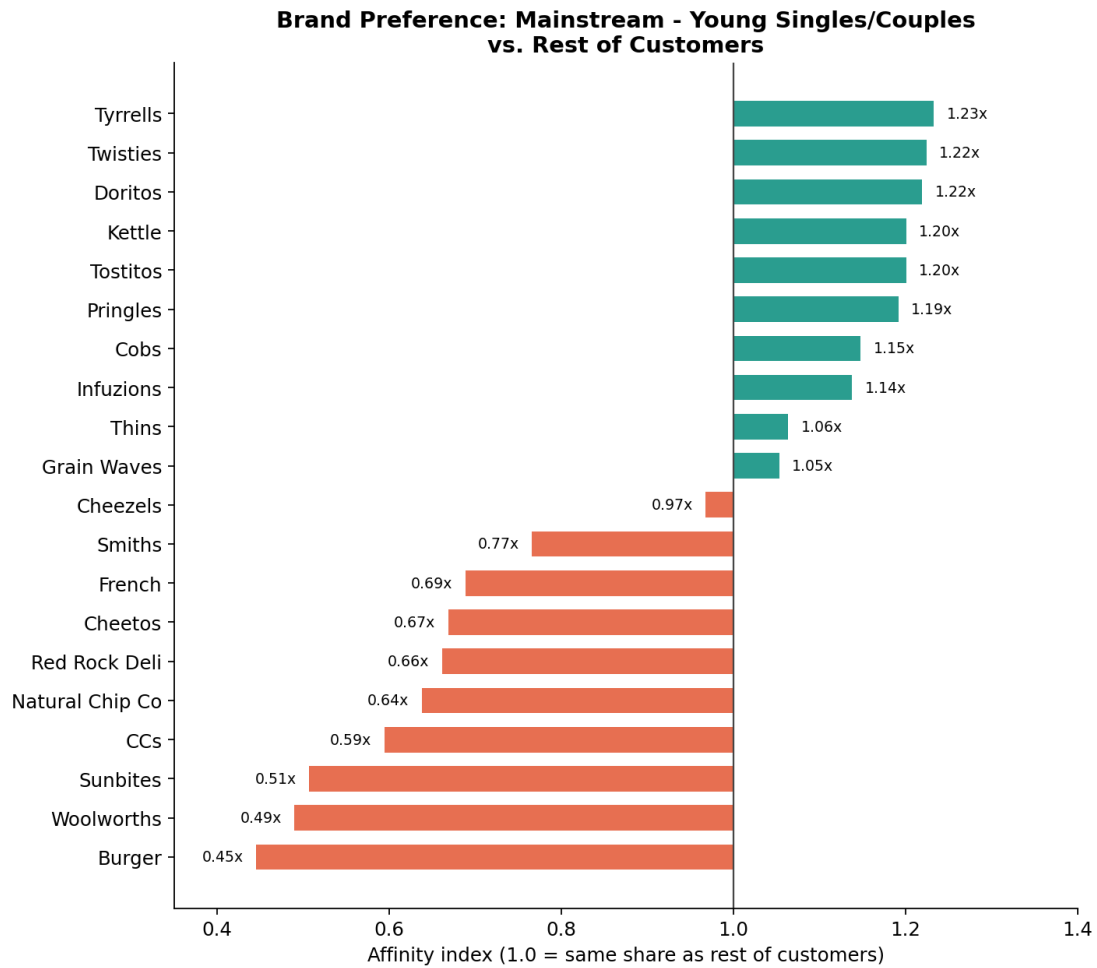


Figure 8: Brand affinity for Mainstream - Young Singles/Couples relative to all other customers.

11 of the 20 brands come in only one pack size (every 134g packet is Pringles; every 270g packet is Twisties). Individual pack-size affinity is therefore mostly a re-statement of the brand affinity above, not an independent finding. Banding into Small/Standard/Large removes the brand confound and shows the genuine size effect is weak:

```
def band(g):
    return "Small (<150g)" if g < 150 else "Standard (150-180g)" if g <= 180 else
        "Large (200g+)"
data["SIZE_BAND"] = data["PACK_SIZE"].apply(band)
# ... repeat the affinity calculation on SIZE_BAND ...
```

Output:

Small (<150g)	1.11x
Large (200g+)	1.07x
Standard (150-180g)	0.94x

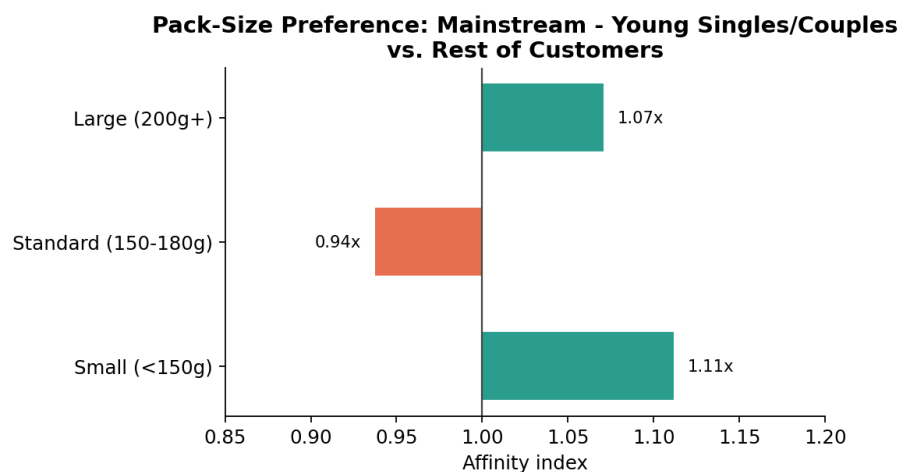


Figure 9: Pack-size affinity for Mainstream - Young Singles/Couples, banded to remove the brand confound.

We can see that:

- This segment buys **slightly less of the standard 150-180g size** (0.94x) and slightly more of both small (<150g, 1.11x) and large (200g+, 1.07x) formats — but all three are within about 10% of parity, a much weaker signal than brand.
- **Pack size is not an independent lever for this segment.** Any size-based promotion should follow from the brand assortment recommended above (e.g. featuring whatever sizes Tyrrells/Doritos/Kettle/Pringles already sell in), rather than leading with a “buy the bigger bag” strategy on its own.

Conclusion and recommendations

Summary of findings

1. **Total sales are \$1,819,778.40** across 475,909 packets and 249,667 transactions from 71,517 customers, and are **stable month-to-month** with one clear pre-Christmas spike.
2. Sales concentrate in **Budget - Older Families, Mainstream - Young Singles/Couples, and Mainstream - Retirees**, but for different reasons: families buy more packets per trip; the Mainstream segments have far more customers.
3. **Mainstream young and midage singles/couples pay a statistically significant ~9% price premium per packet** versus Budget/Premium in the same lifestage ($p < 0.0001$) — notably, Mainstream Retirees do not show this same premium.
4. The target segment (**Mainstream - Young Singles/Couples**) clearly over-indexes on **Tyrrells, Twisties, Doritos, Kettle, Tostitos, and Pringles**, and under-indexes on Woolworths, Burger Rings, Sunbites, CCs, Natural Chip Co, Red Rock Deli, and Cheetos.
5. **Pack size is not an independent driver** for this segment once brand is accounted for — most of the apparent size preference is really brand preference, since most brands are sold in only one size.

Recommendation to Julia

Prioritise **Mainstream - Young Singles/Couples** as the lead target segment for the next six months: it is the single largest customer base in the category, it carries a proven, statistically significant willingness to pay more per packet, and its brand preferences are clear and actionable. Category and promotional space should lean into **Tyrrells, Twisties, Doritos, Kettle, Tostitos, and Pringles** for this segment, rather than spreading investment evenly across all 20 brands — and pack-size strategy should follow the brand assortment rather than lead it. Budget - Older Families and other large-basket family segments merit a separate volume/frequency-based workstream, as their driver (packets per trip) is fundamentally different from the Mainstream Singles/Couples reach-and-premium story above.